

3주 차 보고서

(2025년 1 월 11 일 ~ 2026년 1 월 17 일)

작성자

배주환

이번 주 회의 내용

이번 주 한 일

이번주에는 몸이 안좋아서 별로 공부를 못했다.

캐시, 메모리 모델, cpu 파이프라인에 대해서 공부를 했는데 컴퓨터 구조와 관련된 내용이었다.

캐시라는 중간 저장소가 있는데 cpu가 core가 여러개가 있고 L1캐시, L2캐시 같이 임시저장소같은 것이 있는 걸 볼 수 있다. L1캐시 같은 경우는 용량이 좀 작지만 코어와 가까이 있고, L2캐시는 용량은 좀 더 크지만 코어와 좀 더 멀리 있다. 여기까지 체크했는데 없으면 RAM까지 가서 체크한다.

캐시에는 두가지의 철학이 있는데

첫번째로 Temporal Locality

시간적으로 보면, 방금 주문한 테이블에서 추가 주문이 나올 확률이 높다는 것이다.

두번째로 Spatial Locality

공간적으로 보면, 방금 주문한 사람 근처에 있는 사람이 추가 주문을 할 확률이 높다.

싱글쓰레드를 사용할때는 컴파일러의 개입이 문제가 없었지만 멀티쓰레드를 사용한다면 컴파일러의 개입 때문에 원하는대로 출력이 안될 수도 있다. 그래서 가시성 및 코드 재배치가 중요하다.

캐시와 파이프라인에 대해서 빨래에 대해서 비유를 해보자.

세탁, 건조, 다림질, 빨래날기의 4단계가 있다고 할때 빨래바구니가 여러개 있을때 한 바구니가 끝날때까지 다른 바구니들이 쉬고 있으면 낭비가 심하다. 그래서 현실적으로는 한 단계가 끝나면 다음 바구니를 시작하고 다음 단계로 가는 방법이 효율적이라는 것을 알 수 있다.

하지만 빨래바구니 별로 크기와 중요도가 달라서 각각 단계를 해결하는데 걸리는 시간이 다르다고 가정을 하면 곧이 곧대로 받은 순서대로 실행하기보다는 순서를 효율적인 방식으로 바꾸는 것이 좋을 것이다.

CPU가 이런 방식으로 동작한다고 보면 된다.

atomic 연산에 한해, 모든 쓰레드가 동일 객체에 대해서 동일한 수정 순서를 관찰할 수 있다.

TLS에 대해서도 공부를 했다.

TLS(Thread Local Storage)란 무엇이나면 큰 영역에 해당하는 Heap영역이나 데이터 영역에 매번 갈 필요 없이 TLS에 넣어놓고 사용할 수 있다.

TLS는 스레드마다 사용할 수 있는 전역메모리같은 느낌이다.

LockStack과 LockQueue를 만들어서 동기화 스택과 큐를 만들어봤다.

```
atomic<bool> flag;

int main()
{
    flag = false;

    // flag = true;
    flag.store(true, memory_order::memory_order_seq_cst);

    // bool val = flag;
    bool val = flag.load(memory_order::memory_order_seq_cst);

    // 이전 flag 값을 prev에 넣고, flag 값을 수정
    {
        bool prev = flag.exchange(true);
        //bool prev = flag;
        //flag = true;
    }

    // CAS(Compare-And-Swap) 조건부 수정
    {
        bool expected = false;
        bool desired = true;
        flag.compare_exchange_strong(expected, desired);
    }
}

#include <mutex>

template<typename T>
class LockStack
{
public:
    LockStack() {}

    LockStack(const LockStack&) = delete;
    LockStack& operator=(const LockStack&) = delete;

    void Push(T value)
    {
        lock_guard<mutex> lock(_mutex);
        _stack.push(std::move(value));
        _condVar.notify_one();
    }

    bool TryPop(T& value)
    {
        lock_guard<mutex> lock(_mutex);
        if (_stack.empty())
            return false;

        // empty -> top -> pop
        value = std::move(_stack.top());
        _stack.pop();
        return true;
    }

    void WaitPop(T& value)
    {
        unique_lock<mutex> lock(_mutex);
        _condVar.wait(lock, [this] { return !_stack.empty(); });
        value = std::move(_stack.top());
        _stack.pop();
    }

private:
    stack<T> _stack;
    mutex _mutex;
    condition_variable _condVar;
};
```

```

thread_local int32 LThreadId = 0;

void ThreadMain(int32 threadId)
{
    LThreadId = threadId;

    while (true)
    {
        cout << "Hi! I am Thread" << LThreadId << endl;
        this_thread::sleep_for(1s);
    }
}

int main()
{
    vector<thread> threads;

    for (int32 i = 0; i < 10; i++)
    {
        int32 threadId = i + 1;
        threads.push_back(thread(ThreadMain, threadId));
    }

    for (thread& t : threads)
        t.join();
}

```

```

atomic<bool> ready;
int32 value;

void Producer()
{
    value = 10;

    ready.store(true, memory_order::memory_order_seq_cst);
}

void Consumer()
{
    while (ready.load(memory_order::memory_order_seq_cst) == false)
        ;

    cout << value << endl;
}

int main()
{
    ready = false;
    value = 0;
    thread t1(Producer);
    thread t2(Consumer);
    t1.join();
    t2.join();

    // Memory Model(정책)
    // 1) Sequentially Consistent (seq_cst)
    // 2) Acquire-Release (acquire, release)
    // 3) Relaxed (relaxed)

    // 1) seq_cst (가장 엄격 = 컴파일러 최적화 어지 적음 = 직관적)
    // 가시성 문제 바로 해결! 코드 재배치 바로 해결!

    // 2) acquire-release
    // 딱 중간!
    // release 명령 이전의 메모리 명령들이, 해당 명령 이후로 재배치 되는 것을 금지
    // 그리고 acquire도 같은 변수를 읽는 쓰레드가 있다면
    // release 이전의 명령들이 -> acquire 하는 순간에 관찰 가능(가시성 보장)

    // 3) relaxed (자유롭다 = 컴파일러 최적화 어지 많음 = 직관적이지 않음)
    // 너무나도 자유롭다
    // 코드 재배치도 멋대로 가능! 가시성 해결은 No!
    // 가장 기본 조건(동일 객체에 대한 동일 관전 순서만 보장)

    // 인텔, AMD의 경우 애당초 순차적으로 일관성을 보장을 해서,
    // seq_cst를 써도 별다른 부하가 없음
    // ARM의 경우 꽤 차이가 있다
}

```

다음 주 할 일

Lock free에 대한 공부를 하고 ThreadManager라는 프레임워크를 만들어볼것이다. 다음 주에는 멀티쓰레드에 대한 공부 끝날 예정이다.

특이사항

